Group Web Application Report

Team Collaboration Web App

Team members: [Add your names]
Course / class: [Add course or class]
Date: [Add submission date]

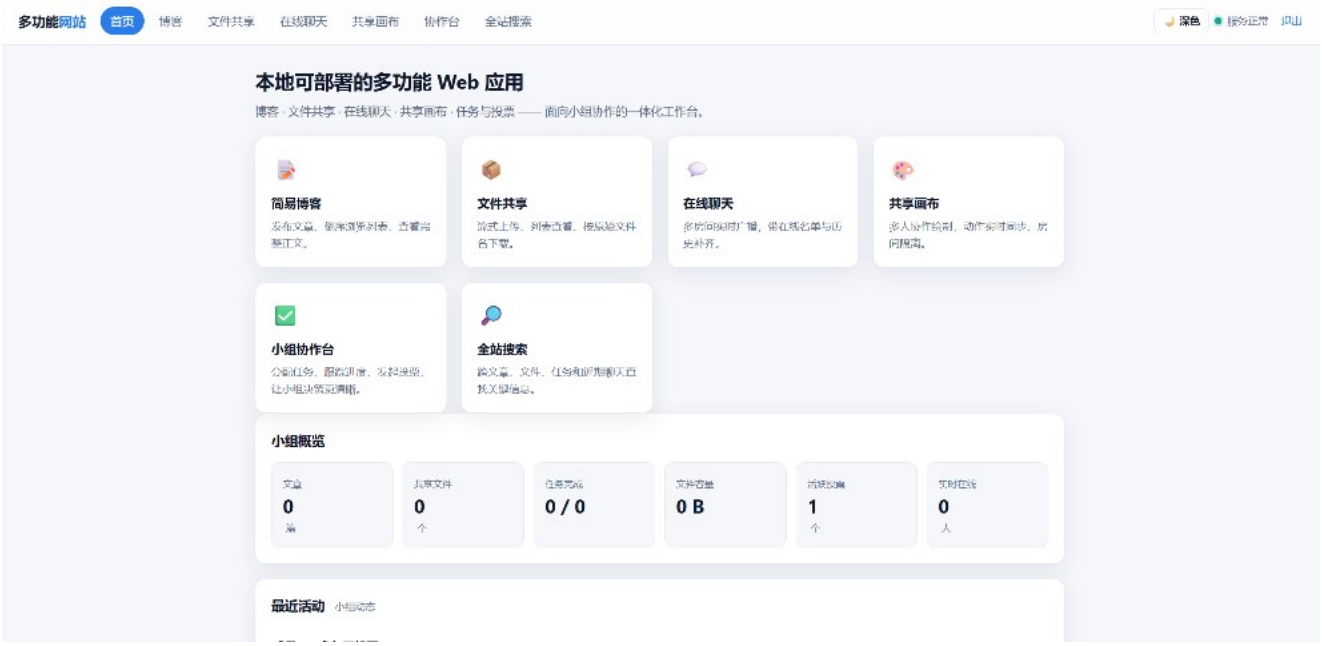
1 Introduction

Our project is a small web application that helps a group share information and work together. It combines communication, file sharing, and simple project planning in one place. The website runs on one computer and can be accessed by group members on the same local network or through a temporary public link.

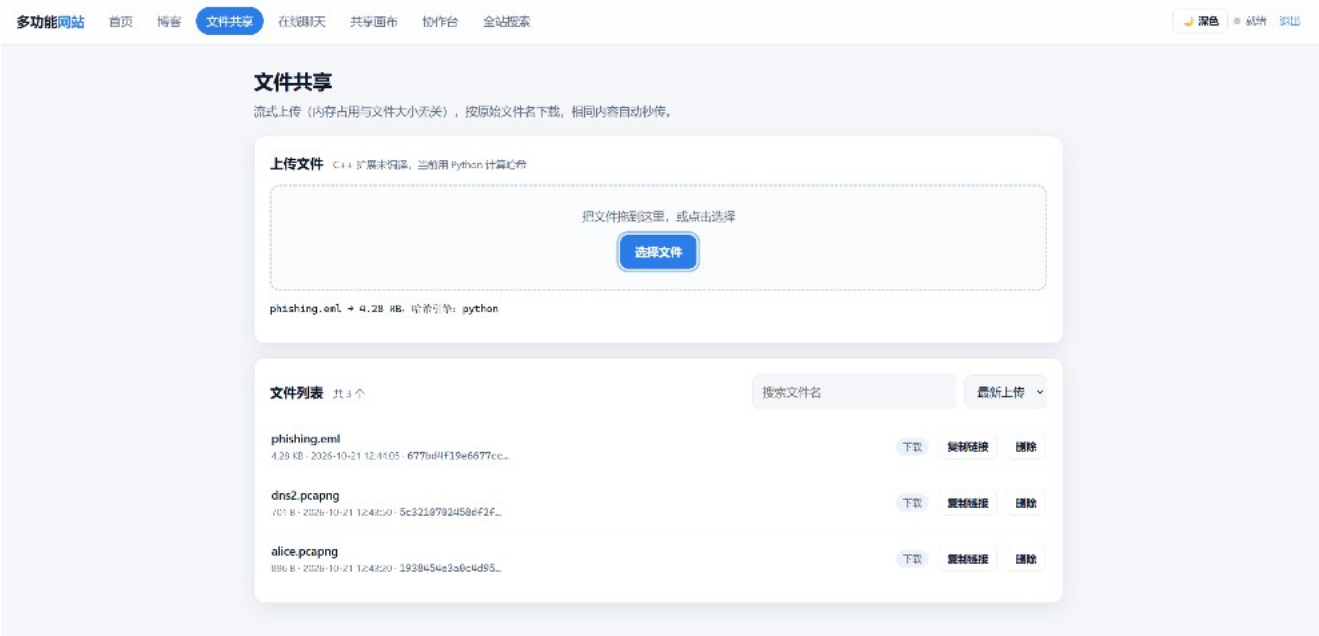
2 Main Features

- **Blog.** Members can publish short posts and read or delete posts.
- **File sharing.** Members can upload, download, and manage shared files.
- **Live chat.** Members can chat in named rooms. WebSocket updates messages without refreshing the page.
- **Shared canvas.** Members in the same room can draw together in real time.
- **Task board.** Tasks can have an assignee, priority, deadline, and status.
- **Polls.** Members can create polls, vote, and change their vote.
- **Search.** Members can search posts, file names, tasks, and recent chat messages.
- **Home overview.** The home page shows summary numbers and recent group activity.

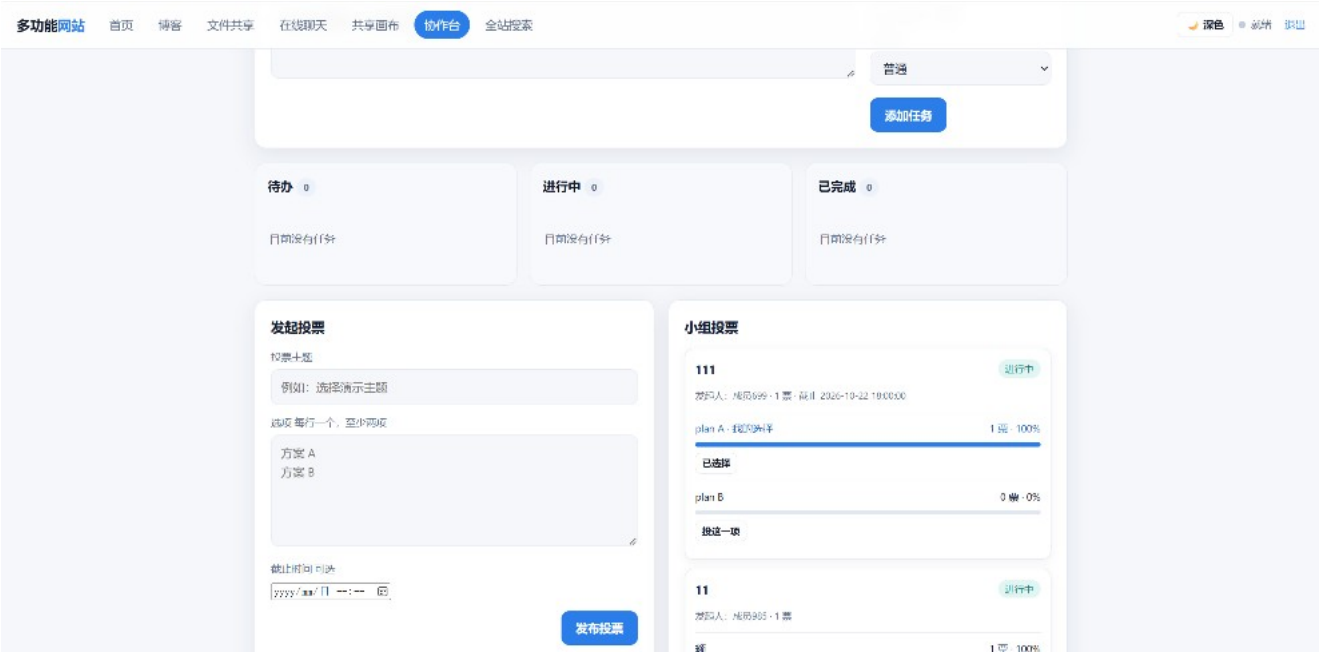
Posts, file information, tasks, polls, and activity records are stored in a local SQLite database. Uploaded files are stored in the project data folder. Recent chat and canvas events are held in memory and are cleared when the server restarts.



Homepage



File sharing



Collaboration and Voting

3 Technologies

The backend uses Python and FastAPI. The frontend uses HTML, CSS, and JavaScript, with no separate build step. SQLite stores project data, and WebSocket supports live chat and drawing. An optional C++ extension can speed up file-related calculations; the application can also run without it.

4 How to Deploy and Run

Windows

The application runs as a single Python service. On Windows, open the project folder and double-click start.cmd, or run the command below in PowerShell.

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts\start.ps1
```

The script creates the virtual environment on first use, installs the packages listed in requirements.txt, loads the optional .env file, starts the server on port 8000 and opens the default browser. The console prints both the local address and the LAN address, for example http://192.168.1.20:8000. Press Ctrl+C to stop the server. The SQLite database and the uploaded files stay in backend/data and backend/uploads, so the data survives a restart.

To start the service manually instead:

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m uvicorn backend.app.main:app --host 0.0.0.0 --port 8000
```

On the first visit the site asks for a nickname. The nickname is kept in a signed cookie for 30 days and is reused by every module (chat, canvas, tasks, polls, posts and files). There is no password and the nickname is not verified. On Linux or macOS the same steps work through start.sh.

Public Access with a Cloudflare Tunnel

The service listens on 0.0.0.0:8000, so other devices in the same network can reach it directly. To reach it from the internet without a public IP address or router port forwarding, we publish it through a Cloudflare quick tunnel. The tunnel keeps an outbound connection to Cloudflare, so no inbound firewall rule is needed.

The steps are:

1. Keep the local server running, as described above.
2. Put cloudflared.exe in .tools\cloudflared.exe, or run scripts\public_tunnel.ps1, which downloads the binary automatically on first use.
3. Start the tunnel with the command below.

   ```
   .tools\cloudflared.exe tunnel --url http://127.0.0.1:8000
   ```

4. Copy the address printed in the banner, for example https://stretch-atom-groundwater-learned.trycloudflare.com, and open it on any device, including a phone using mobile data.
5. Because the public page is served over HTTPS, the WebSocket connections switch to wss:// automatically, so live chat and the shared canvas keep working through the tunnel.

A quick tunnel is meant for demonstrations. The host name is random, it changes every time the tunnel restarts, and Cloudflare can close a tunnel that stays idle; the log then shows “Unauthorized: Tunnel not found” and the old address stops resolving. Restarting the command produces a new address. A stable address needs either a named tunnel bound to a domain (cloudflared tunnel login, tunnel create, tunnel route dns) or a fixed-name service such as Tailscale Funnel.

Security note. Anyone who receives the tunnel link can open the site. The current sign-in only records a nickname and does not authenticate the visitor, so a public tunnel should not carry private material unless real authentication is added.

Configuration

Optional settings live in .env in the project root and are loaded by scripts/start.ps1. APP_HOST and APP_PORT select the listen address, APP_MAX_UPLOAD_BYTES caps the upload size, APP_SECRET signs the identity cookie

(changing it signs every device out), and APP_CORS_ORIGINS restricts cross-origin access. All keys and their defaults are documented in .env.example.

5 Testing and Debugging

We tested the main pages and APIs, including task creation, polls, statistics, search, and login. The automated test suite completed with 38 tests passed and 1 skipped.

During development, the browser could load the new collaboration pages while the running server still returned not found for their APIs. An older server process was still using the previous application code. Restarting the server loaded the new routes, and the pages and APIs then worked as expected.

Debugging: the sign-in page kept coming back

After we replaced the shared password with a nickname-based sign-in, opening the site kept returning to the same sign-in screen. The page reloaded continuously, so the input field could not be used at all.

Cause. Two problems combined. First, the browser still had the previous sign-in page and its script in the cache. That script requested /api/session and read a field named auth_enabled, which the new backend no longer returns. Reading undefined was treated as “no password is configured”, so the script redirected to the requested page; the server saw no identity and redirected back to the sign-in page, and the two sides kept bouncing. Second, the static files were served with only ETag and Last-Modified headers and no Cache-Control, so the browser was free to keep using the stale copy.

Resolution. We removed the shared password altogether and adopted a simpler model: the first visit asks for a nickname, which is stored in a signed HttpOnly cookie for 30 days and reused everywhere. The nickname is a convenience for identifying who wrote or uploaded something, not a credential, so no password is needed. On the technical side, the sign-in script now redirects only when the API explicitly returns authenticated: true; HTML is served with Cache-Control: no-store, scripts with no-cache, and the sign-in assets carry a version query, so a cached page cannot survive a deployment. If a browser is still stuck on the old page, opening the address with a cache-busting query string, or with a freshly created tunnel address, forces a clean load.

6 Team Contributions

Replace the examples below with your group's actual work.

- [Member name]: [Describe their contribution, such as frontend pages or styling.]
- [Member name]: [Describe their contribution, such as backend APIs or database work.]
- [Member name]: [Describe their contribution, such as testing, debugging, or documentation.]